RV Institute of Technology and Management®

Department of Master of Computer Applications

# SOFTWARE REQUIREMENTS SPECIFICATION AND HIGH-LEVEL DESIGN

### AI Emergency Response Platform

**Status note.** This document describes the *proposed* system. Every feature, technology and component named here is planned for the project; none is claimed as implemented. The AI model or service will be selected after comparing options (rule-based baseline, TF-IDF with a machine-learning classifier, and a pretrained language-model API).

## Part A: Software Requirements Specification (SRS)

### 1. Purpose and Scope

The AI Emergency Response Platform is a web application where users report emergencies or incidents with a description, an optional image and a location. An AI component recommends the incident category and priority. The report is routed to an authorised responder, who handles it, and the user can track its progress. Administrators monitor all incidents. The system is a college demonstration: AI output is only a recommendation that a human verifies, and the system does not contact real emergency services.

**Out of scope:** contacting police, fire or ambulance services; replacing official emergency numbers; SMS, email or push notifications (planned only as future work).

### 2. User Roles and Permissions

| Action | User | Responder | Admin |
|---|---|---|---|
| Register, log in | Yes | Log in only (account created by Admin) | Log in |
| Submit an incident report | Yes | No | No |
| View own reports and status | Yes | No | Yes |
| View all incidents | No | Assigned or own category only | Yes |
| Accept incident, update status, add notes, resolve | No | Yes | Can update on behalf if needed |
| Assign or reassign incidents | No | No | Yes |
| Correct AI category or priority | No | Can review and flag | Yes |
| Manage users and responders | No | No | Yes |
| View analytics and filters | No | No | Yes |

Example: a User who types an admin page address is refused by the backend (not only hidden in the interface).

### 3. Functional Requirements

| ID | Requirement | Example from this project |
|---|---|---|
| FR-01 | Registration and login with name, email and password. Passwords are stored only as hashes; a JWT is issued on login. | A new student registers and logs in to see the User dashboard. |
| FR-02 | Role-based access for User, Responder and Admin, enforced on every protected API route. | A Responder cannot open the Admin analytics API. |

| ID | Requirement | Example from this project |
|---|---|---|
| FR-03 | Submit an incident report. Description is required; image and extra details are optional. | “Smoke from the second floor of Block B and people are inside.” |
| FR-04 | Capture location by current GPS, by clicking on a map, or by typed description. GPS permission is not mandatory. | User denies GPS and types “Near Block B main entrance”. |
| FR-05 | Optional image upload (JPG, JPEG, PNG) with size limit, preview and remove option. | A 12 MB image is rejected with a clear message. |
| FR-06 | AI analysis returns category, priority, short summary and reason; shown as a recommendation. | Category Fire, priority High, reason: smoke and people inside. |
| FR-07 | Route the report to the responder group for the category; if none is available, set Pending Assignment and notify the Admin. | No Fire responder online, so Admin is notified. |
| FR-08 | Responder can view details, image and map location, accept, start work, add notes and resolve. | Fire responder accepts the incident and adds a note. |
| FR-09 | Status flow: Submitted, AI Analyzed, Assigned, Accepted, In Progress, Resolved. Only valid transitions are allowed; every change is saved with who, when and an optional note. | A client cannot jump from Submitted to Resolved. |
| FR-10 | User can track a report on a timeline and see report history. | User sees AI Analyzed and Assigned ticked, In Progress current. |
| FR-11 | In-app notifications for users, responders and admins on key events. | User gets “Responder assigned”. |
| FR-12 | Admin can view all incidents, filter by date, category, priority and status, assign incidents, manage users and correct AI classifications. | Admin changes category from Other to Infrastructure. |
| FR-13 | Admin dashboard with totals and charts (by category, priority, status, time). | Chart shows incidents per category. |
| FR-14 | If the AI service fails or returns invalid output, the report is still saved and goes to manual review. | AI timeout: report stays Submitted and is flagged. |

## 4. Non-Functional Requirements

| Requirement | Description | Example |
|---|---|---|
| Security | Password hashing, JWT verification, server-side validation, safe file checks, no secrets in frontend code. | The AI API key stays in backend environment variables. |
| Privacy | Users can read only their own reports; location and images are not shown in public views. | User A cannot open User B’s report by changing the URL. |
| Usability | The report form is short and fast to complete, with clear validation messages. | An urgent report needs only description and location. |
| Responsive design | Works on desktop and mobile browsers. | A user reports from a phone browser. |
| Reliability | The report is saved before AI analysis starts, so it is not lost if the AI fails. | AI outage does not block submission. |
| Performance | Pages load quickly and AI calls have a fixed timeout so the interface never waits indefinitely. | Submission returns a report ID immediately. |
| Maintainability | Separate folders for frontend, routes, controllers, models, middleware and AI logic; AI behind an adapter. | The AI model can be swapped without changing other modules. |
| Scalability and availability | Stateless API (JWT) and a document database allow more users and later hosting changes. | More responders can be added without code changes. |
| Auditability | Every status change is stored in an incident history. | Admin can see who resolved an incident and when. |

Difference: a functional requirement says *what* the system does (FR-06: classify the report); a non-functional requirement says *how well or under what conditions* it does it (the classification call must time out safely).

## 5. Input, Processing and Output

| Input | Processing | Output |
|---|---|---|
| Registration and login details | Validation, password hashing, JWT creation | Account created; token; role dashboard |
| Description, optional image, location (GPS, map point or text) | Validation, file checks, storage, report ID, AI analysis, routing | Report ID, AI recommendation, assigned responder |
| Responder actions: accept, status, notes | Transition checks, history entry, notification creation | Updated status, timeline, notifications |
| Admin filters, assignments, corrections | Queries, aggregation, role checks | Filtered lists, charts, updated assignments |

## 6. Software and Hardware Requirements

| Category | Requirement (proposed) |
|---|---|
| Frontend | React.js, JavaScript ES6+, Bootstrap 5, React Router, Axios or Fetch API |
| Backend | Node.js (LTS) and Express.js, JWT, password hashing library, file upload middleware |
| Database | MongoDB with Mongoose (local installation or cloud-hosted) |
| AI | Model or API to be selected after comparison; called only from the backend |
| Map and location | Browser geolocation and an interactive map library (to be selected) |
| Tools | VS Code, Git, npm, Postman or similar API testing tool, modern web browser |
| Development computer (suggested) | Windows, Linux or macOS; 8 GB RAM recommended; multi-core processor; about 5 GB free disk; internet connection |
| User device | Any device with a modern browser and internet; GPS optional |

## 7. Assumptions, Constraints and Limitations

- Users have an internet connection; GPS permission is optional and a manual location is always accepted.
- Reports are assumed to be written mainly in English for AI analysis.
- Responders and admins are internal accounts created for the demonstration; there is no connection to real emergency departments.
- AI recommendations may be wrong or uncertain, so a human verifies them. Confidence values are shown only if the chosen model provides a meaningful one.
- AI API limits, cost and response time constrain the design; image analysis depends on the chosen model.
- Limited academic timeline: notifications are in-app only; email, SMS and push are future work.

## Part B: High-Level Design (HLD)

HLD shows the big picture: which main components exist and how they talk to each other. It does not show code-level details. For example, HLD says “the backend sends the report text to an AI Analysis Adapter and stores the result”; LLD (next document) will define the exact fields, API routes and validation rules.

### 1. Architecture Diagram

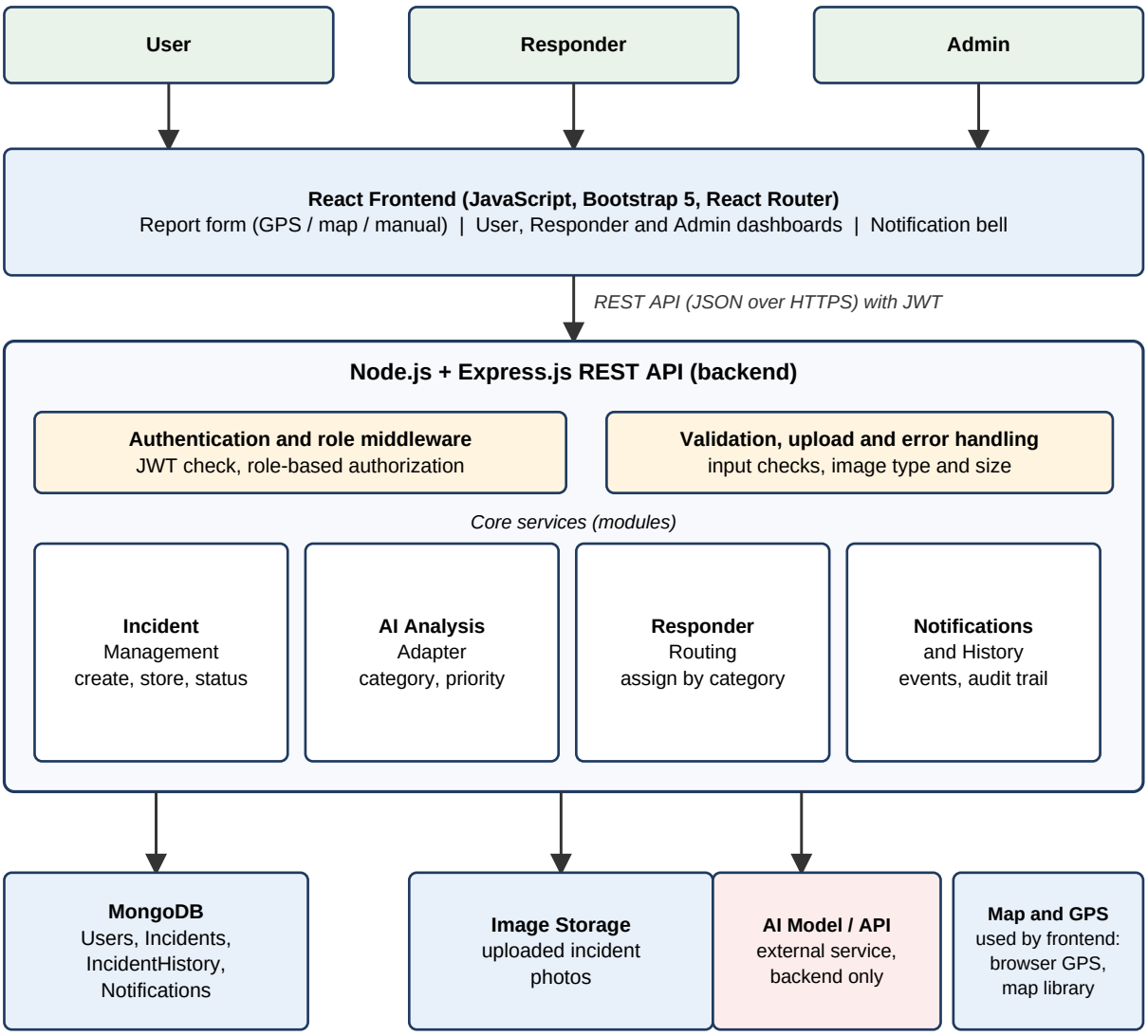



Figure: High-level architecture of the AI Emergency Response Platform (proposed). Map and GPS are used directly by the frontend, so they have no arrow from the backend; they are shown at the bottom with the other supporting services.

### 2. Main Components and Responsibilities

| Component | Responsibility | Example |
|---|---|---|
| React frontend | Shows forms and dashboards, validates input on the client, keeps the login token, calls the REST API. | User, Responder and Admin screens |
| Express REST API | Receives requests, applies middleware, runs the business logic and returns JSON. | POST /api/incidents |
| Auth and role middleware | Verifies the JWT and checks the role before a route runs. | Blocks a User from admin routes |
| Incident Management | Creates the report ID, saves the incident and controls valid status changes. | Saves a Fire report as Submitted |

| Component | Responsibility | Example |
|---|---|---|
| AI Analysis Adapter | Sends report text (and image if supported) to the AI model, validates the answer, returns category, priority, summary and reason. Never changes the database or permissions itself. | Rejects a category not in the allowed list |
| Responder Routing | Maps the category to a responder group and assigns; sets Pending Assignment if none is available. | Fire goes to the Fire responder group |
| Notification and History | Stores every status change and creates in-app notifications. | “Your incident is In Progress” |
| MongoDB | Stores users, incidents, incident history and notifications. | Incident document with AI fields |
| Image storage | Keeps uploaded photos; the database stores only the image path. | photo of smoke at Block B |
| Map and geolocation | Browser geolocation gives latitude and longitude; the map lets users pick or view a point. | User taps a point on the map |

## 3. Data Flow: Report Submission to Resolution

| Step | What happens |
|---|---|
| 1 | User fills the report form (description, optional image, location) in the React app. The frontend validates the input. |
| 2 | The frontend sends the report to the REST API with the JWT token. |
| 3 | Auth middleware verifies the token and role; validation middleware checks the input and image. |
| 4 | The backend creates a report ID, stores the image, and saves the incident with status Submitted. |
| 5 | The AI Analysis Adapter sends the text (and image if supported) to the AI model; the backend validates the reply (allowed category and priority only). |
| 6 | The recommendation (category, priority, summary, reason) is saved and the status becomes AI Analyzed. If the AI fails, the report stays Submitted and is flagged for manual review. |
| 7 | Routing finds the responder group for the category. If available, status becomes Assigned and the responder is notified; otherwise Pending Assignment and the Admin is notified. |
| 8 | The Responder reviews the report, accepts it, starts work, adds notes and finally resolves it; each change follows the allowed status flow. |
| 9 | Every status change is saved in the history and a notification is created for the user. |
| 10 | The User tracks progress on the timeline; the Admin sees all incidents and analytics. |

## 4. How the Components Communicate

- Frontend to backend: HTTPS requests with JSON data (REST). The JWT token is sent with each protected request.
- Backend to MongoDB: through Mongoose models; the frontend never talks to the database.
- Backend to AI model: an outgoing API call made only from the backend, so the API key is never exposed to the browser.
- Backend to image storage: uploaded files are checked, stored, and referenced by path in the incident record.
- Browser to map and GPS: handled in the frontend; the resulting latitude and longitude are sent with the report.

## 5. Why This Architecture

- A client-server REST design separates the interface from the logic, so each can be developed and tested separately.
- JWT makes the API stateless and supports role-based access for three kinds of users.
- Putting AI behind an adapter lets us compare or replace models without changing the rest of the system, and lets the backend validate every AI answer.

- MongoDB documents fit incident records whose fields (location source, optional image, AI fields) vary between reports.

## 6. Assumptions and External Dependencies

- The chosen AI service must be reachable and within its usage limits; the system continues without it (manual review).
- Browser geolocation depends on the user’s permission and device; the map library may depend on an online tile provider.
- MongoDB (local or cloud) and the hosting environment must be available during the demonstration.
- All responder accounts are internal demonstration accounts.